Process Mining

Chiara Di Francescomarino
a.y. 2025/2026
University of Trento

Davide Donà
Andrea Blushi

January 2026

Notes Repository

Contents

1 Business Process Model and Notation (BPMN)

1.1 BPMN Syntax

BPMN is a graphical representation for specifying business processes in a business process model. It provides a standard notation that is readily understandable by all business stakeholders, including business analysts, technical developers, and business managers.

1.1.1 Core Elements

The core elements of BPMN can be categorized into four main groups:

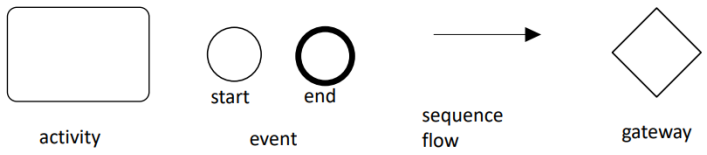


Figure 1: Core Elements of BPMN

1. **Activities**: represent work that is performed within a business process.
2. **Events**: represent something that happens during the course of a business process. The core types of events are:
   - **Start Event**: trigger a new process instance, by generating a token to start the process;
   - **End Event**: signals that a process instance has completed its execution, by consuming a token.
3. **Sequence Flows**: represent the order in which activities and events are executed within a business process.
4. **Gateways**: represent decision points in a process where the flow can diverge or converge based on certain conditions. The core types of gateways are:
   - **XOR Gateway (Exclusive Gateway)**: represents a decision point where only one of the outgoing paths can be taken based on conditions. The **XOR-split** takes one outgoing branch, while the **XOR-join** proceeds when one incoming branch completes.

Figure 2: XOR-split and XOR-join Gateways

- **AND Gateway (Parallel Gateway)**: provides a mechanisms to create and synchronize parallel paths in the process flow. The **AND-split** creates multiple concurrent paths (all outgoing branches are taken), while the **AND-join** synchronizes multiple incoming paths (waits for all incoming branches to complete before proceeding).

Figure 3: AND-split and AND-join Gateways

- **OR Gateway**: provide a mechanism to create and synchronize m out of n paths in the process flow. The **OR-split** can take one or more outgoing branches, while the **OR-join** waits for all the active incoming branches to complete before proceeding.

Figure 4: OR-split and OR-join Gateways

1.1.2 Naming conventions

It is important to follow naming conventions for clarity:

- **Event**: Use noun + past participle (e.g., "Order Received").
- **Activity**: Use verb + noun (e.g., "Approve Invoice").
- **Split Gateway**: Use descriptive labels with conditions(e.g., "Is Payment Approved?").

1.1.3 Data Objects

A **Data Object** captures an artifact required (input - the arrow pointing towards the activity) or produced (output - the arrow pointing away from the activity) by activities within a process. The **status** of the business object can be denoted inside square brackets (e.g., [approved], [pending]).

An **association link** (dashed line) is used to connect data objects to activities or events of business processes. All occurrences of a Data Object represent the same underlying data.

It's important to note that **input data objects** represent a blocking condition for the activity to start (the activity cannot start until the input data object is available).

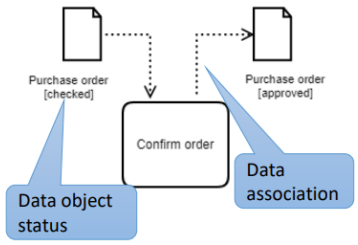


Figure 5: Data Object in BPMN

1.1.4 Data Store

A Data Store is a place containing data objects that is persistent beyond the lifetime of a process instance.

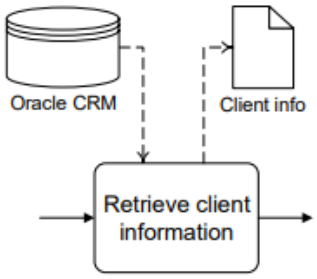


Figure 6: Data Store in BPMN

1.1.5 Text Annotations

A Text Annotation is used to add descriptive information to a BPMN diagram. It provides additional context or explanations about specific elements or the overall process flow. Doesn't affect the process flow.

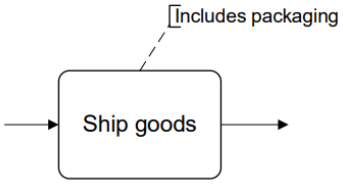


Figure 7: Text Annotation in BPMN

1.1.6 Pools and Lanes

Pools captures a resource class. Generally used to model a business party (e.g., a whole company, clients or suppliers).

Lanes are subdivisions within a pool that represent specific roles, departments, or functions within the business party. They can be seen as a resource subclass, within a pool.

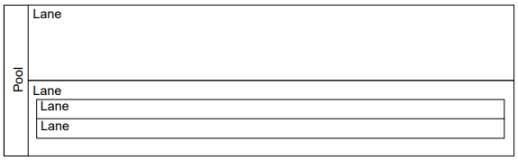


Figure 8: Pools and Lanes in BPMN

Activity that are placed within a lane indicates that it must be performed by the role or department represented by that lane.

1.1.7 Message Flow

A Message Flow represents the flow of messages between two separate process participants (pools). It indicates communication or interaction between different entities in a business process. A Message Flow can connect:

- directly the boundary of a pool (from a party to another party)
- to a specific activity or event within a pool (from a party to an activity/event or vice versa)

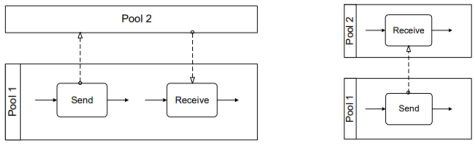


Figure 9: Message Flow in BPMN

These elements have some syntactic rules:

- A **sequence flow** (solid line) cannot cross pool boundaries; it can only connect elements within the same pool;
- Both **sequence** and **message flows** can cross the boundaries of lanes within a pool;
- A **message flow** cannot connect two flow elements within the same pool;

An **incoming message flow** may lead to the creation of the data object by the activity receiving the message.

1.2 Advanced BPMN Constructs

1.2.1 Sub-processes

In BPMN, a sub-process is a compound activity that encapsulates a set of related tasks or activities within a larger process. It **requires** a **start event** and an end **event to define** its boundaries.

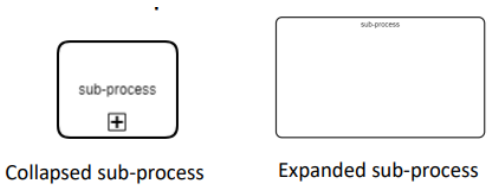


Figure 10: Sub-process in BPMN

It can also delimit parts of the process that can be repeated or interrupted as a whole.

1.2.2 Block-structured repetition

Activity loop markers can be used to denote that an activity or sub-process can be repeated multiple times, based on certain conditions. It can be used when we have a single entry and a single exit point for the repeated section of the process. Then the completion condition is explicitly defined with a **textual annotation** (1.1.5).

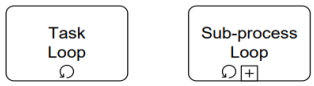


Figure 11: Activity Loop Marker in BPMN

1.2.3 Parallel repetition: multi instance activity

The multi-instance marker indicates that an activity or sub-process can be executed multiple times concurrently. Useful when the same activity need to be performed for multiple instances of a data object (e.g., processing multiple orders).

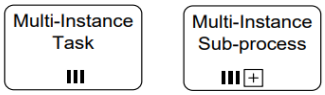
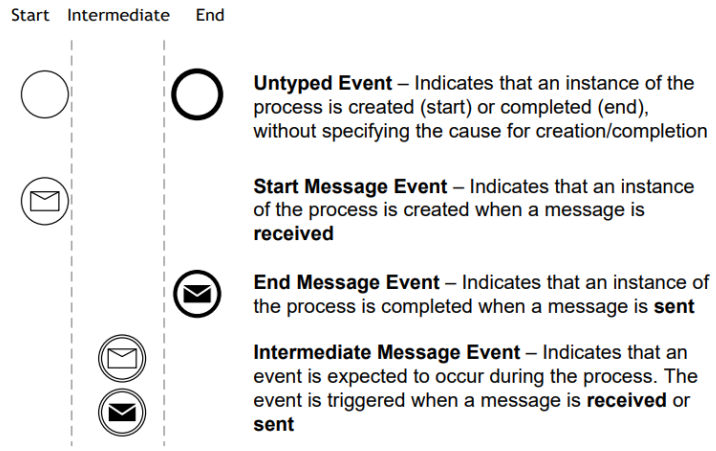


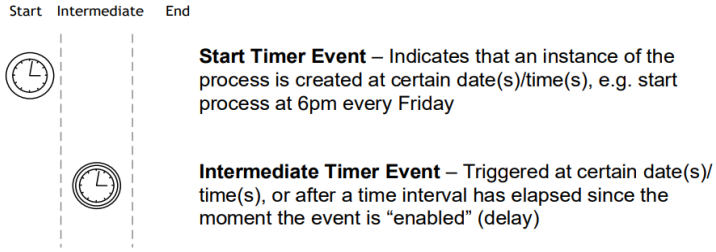
Figure 12: Multi-instance Marker in BPMN

We can also use a textual annotation to specify the number of instances or the conditions for creating new instances. Then we can use this notation also for pools and lanes to indicate that multiple instances of a resource class are involved in the process.

1.2.4 Events

Events in BPMN represent something instantaneous that happens during the execution of a process. They are classified as follows:





Untyped events: they are the default events (start and end), already introduced in the core elements.

Message events: represent the sending or receiving of a message between two process participants (pools).

- **Message Start Event**: triggers a new process instance upon receiving a message;
- **Message End Event**: indicates that a message is sent when the process instance completes;
- **Message Intermediate:** can be used to send (black envelope) or receive (white envelope) messages during the execution of a process instance.

It's important to use message events only when the corresponding activity would simply send or receive a message without any additional processing. Message receive events should be used to model the receipt of messages that trigger the start of a process or an intermediate event within a process, and not to model the receipt of messages that are part of the normal flow of activities within a process.

Temporal events: represent time-based triggers or delays within a process.

- **Timer Start Event**: triggers a new process instance based on a specific time or date. This is usually used for process that need to start at specific times (e.g., 6pm every friday);
- **Timer Intermediate Event**: can be used to introduce delays or wait for a specific time during the execution of a process instance. Usually used to model timeouts or waiting periods within a process flow.

1.2.5 Event-based decision

Sometimes, a decision point in a process is based on the occurrence of specific events rather than conditions evaluated on data. In such cases, an event-based gateway is used to model the decision point. It works similarly to an XOR gateway (only one branch can be taken), but instead of evaluating conditions on data, it waits for one of the specified events to occur before proceeding along the corresponding path.

Figure 13: Event-based Gateway in BPMN

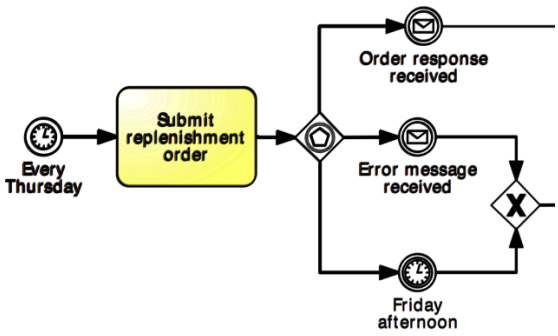


Figure 14: Example of Event-based Gateway in BPMN

1.2.6 Exception Handling

In BPMN, exception handling is used to manage unexpected events or errors that may occur during the execution of a business process. BPMN provides several constructs to model exception handling, allowing processes to gracefully handle errors and continue execution or terminate as needed.

Abortion: The simplest way to handle exceptions is to abort the process instance when an exception occurs. This can be done via the Terminate End Event, which immediately ends the process instance regardless of its current state. This causes the abortion of all ongoing activities and the termination of the process instance.

Figure 15: Abortion of a Process Instance in BPMN

We can use more sophisticated exception handling mechanisms, in order to recover from exceptions and continue the process flow.

Boundary Events Through boundary events, we catch intermediate events on the boundary of an activity or sub-process. When the boundary event is triggered, the normal flow of the process is interrupted, and the exception handling flow is initiated. There

- **External boundary events**: triggered by events external to the process, the execution of the current activity must be interrupted. Handled with intermediate message events;

- **Internal boundary events**: something goes wrong inside an activity, whose execution must be interrupted. Handled with intermediate error events;

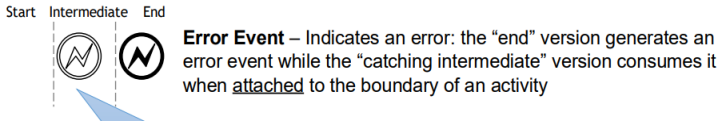


Figure 16: Internal Boundary Event in BPMN

- **Timeout boundary events**: an activity takes too long to complete, and its execution must be interrupted. Handled with intermediate timer events.

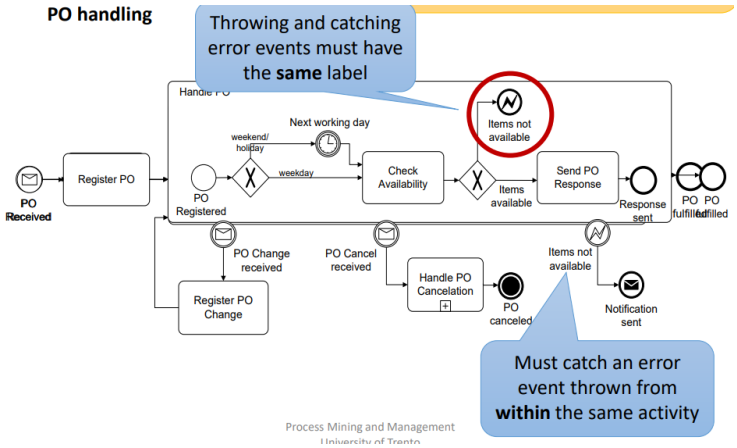


Figure 17: Example of Boundary Events in BPMN

Non-interrupting Boundary Events Sometimes, we want to handle exceptions without interrupting the normal flow of the process. In such cases, we can use non-interrupting boundary events. When a non-interrupting boundary event is triggered, the exception handling flow is initiated, but the normal flow of the process continues concurrently.

Figure 18: Non-interrupting Boundary Event in BPMN

Compensation Events Compensation events are used to handle situations where an activity needs to be undone or compensated for after it has been completed. A compensation event is typically associated with a specific activity or sub-process, and it defines the actions that need to be taken to reverse or mitigate the effects of that activity.

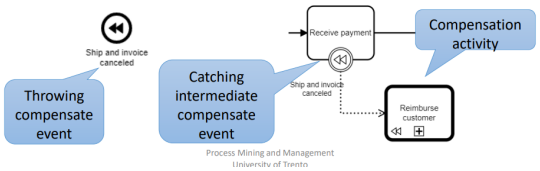


Figure 19: Compensation Event in BPMN

2 Process Analysis

Process analysis aims to evaluate the performance and efficiency of business processes, to guide process improvement initiatives. Taking as input a *as-is* process model, it provides insights on weaknesses and their impact. We can distinguish between two main types of analysis:

- **Qualitative Analysis**: focuses on identifying inefficiencies, bottlenecks, and areas for improvement within a process.
- **Quantitative Analysis**: focuses on measuring and evaluating the performance of a process using various metrics.

2.1 Process Qualitative Analysis

2.1.1 Value-added analysis

This kind of analysis focuses on **identifying and categorizing** activities within a business process based on their contribution to the overall value delivered to customers or stakeholders. After that we can optimize the process by **eliminating** non-value-adding activities.

Identification We break down the process into atomic steps, examining each one of them to determine its category among the following:

- **Value-added activities**: directly contribute to delivering value or satisfaction to the customer
  - Is the customer willing to pay for this activity?
  - Would the customer agree that this step is necessary to achieve their goals?
  - If the step were eliminated, would the customer notice a difference in the final product or service?
- **Business value-added activities**: necessary for the business operation but do not directly add value from the customer's perspective
  - Is this step required in order to collect revenue, to improve or grow the business?
  - Would the business potentially suffer if this step were eliminated?
  - Does it reduce risk of business losses?
  - Is this step required to comply with regulations or legal requirements?
- **Non-value-adding activities**: do not add value and are not necessary for the business (everything that is not VA or BVA). This usually includes handovers, delays, rework, etc.

Waste Elimination After identifying and categorizing the activities, we can remove NVA activities and carefully evaluate BVA activities to see if they can be optimized or eliminated without negatively impacting the business. We have different sources of waste:

- **Move**:
  - **Transportation**: send or receive materials or document taken as input or output of an activity;
  - **Motion**: movement of resources (people, materials) internally within the process.
- **Hold**:
  - **Inventory**: accumulation of materials or documents waiting to be processed (Work in Progress - WIP);
  - **Waiting**: task waiting for input data or resource availability, or resources waiting for task assignment.
- **Over-do**:
  - **Defects**: correcting or compensating for errors or mistakes in the process (rework loops);
  - **Over-processing**: tasks performed unnecessarily given the outcome required by the customer;
  - **Over-production**: unnecessary process instances are performed, producing outcomes that do not add value upon the completion of the process.

2.2 Process Quantitative Analysis

The process performance can be measured through different dimensions:

- **Time**: how long it takes to complete a process or specific activities within the process;
- **Cost**: the expenses incurred to execute the process, including labor, materials, and overhead costs;
- **Quality**: the degree to which the process meets customer requirements and expectations.

2.2.1 Time Measures

Cycle (throughput) time : the time between start and completion of a process instance. It can be measured as:

$$\text{Cycle throughput time} = \text{Processing Time (PT)} + \text{Waiting Time (WT)} \quad (2.1)$$

where:

- **Processing Time (PT)**: time taken by value-adding activities
- **Waiting Time (WT)**: time spent in non-value-adding activities

The result can be different from a simply sum, depending on the process structure (e.g., parallel paths, loops, etc.).

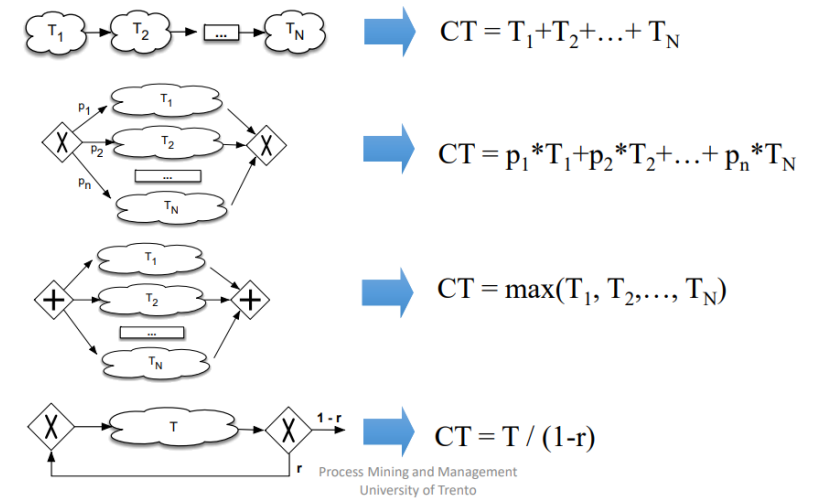


Figure 20: Cycle Time Calculation Table

- In case of **sequential** activities, the cycle time is the sum of the cycle times of each activity;
- In case of **exclusive choices** (XOR), the cycle time is the weighted average of the cycle times of each branch, using the branching probabilities as weights;
- In case of **parallel** activities, the cycle time is the maximum cycle time among the parallel branches;
- In case of **loops**, the cycle time is the sum of the cycle time of the activities inside the loop, divided by the probability of exiting the loop.

Cycle Time Efficiency Given the cycle time, we can calculate the efficiency of the process. The Cycle Time Efficiency (CTE) is a measure of how efficiently a process is executed, focusing on the proportion of time spent on value-adding activities compared to the total cycle time. It is calculated as:

$$\text{CTE} = \frac{\text{Processing Time (PT)}}{\text{Cycle throughput time}} \tag{2.2}$$

2.2.2 Cost Measures

Per-instance Cost The per-instance cost of a process measures the total cost incurred to complete a single instance of the process. It is calculated as:

$$\text{Per-instance Cost} = \text{Processing cost} + \text{Cost of waste} \tag{2.3}$$

where:

- **Processing cost**: cost associated with value-adding activities
- **Cost of waste**: cost associated with non-value-adding activities

2.2.3 Work-In-Progress (WIP)

Work-In-Progress (WIP) refers to the number of process instances that are currently being executed but have not yet been completed. WIP is a form of waste. Using Little's Law, we can relate WIP to the arrival rate and cycle time:

$$\text{WIP} = \lambda \times \text{Cycle Time} \tag{2.4}$$

where λ is the arrival rate (number of cases arriving per time unit). Its limitation is that we can't estimate waiting times or queue lengths.

3 Petri Nets

A Petri Net is a business process modeling notation, based on some strong mathematical foundations. They offer a formal way to represent processes, allowing for rigorous analysis and verification of process properties.

3.1 Unformal introduction to Petri Nets

Elements of a Petri Net: A Petri Net consists of the following elements:

- **Places**: represent the states or conditions of the system;
- **Transitions**: represent the events that can change the state of the system;
- **Tokens**: represent the presence of a condition or the resources required for a transition to occur;
- **Arcs**: connect places to transitions and transitions to places, indicating the flow of tokens.

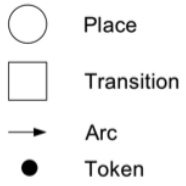


Figure 21: Basic Petri Net Structure

Syntactic Rules: A Petri net can be defined as a **bipartite directed graph** with two types of nodes: places and transitions. The arcs between these nodes indicate the flow of tokens and define the behavior of the system.
The **state** of a Petri net (**marking**) is represented by the distribution of tokens across its places.

- A transition is **enabled** if all its input places contain the required number of tokens;
- When an enabled transition **fires**, it consumes tokens from its input places and produces tokens in its output places.

3.2 Formal Definition of Petri Nets

A Petri net is formally defined as a **triple** $PN = (P, T, F)$ where:

- P is a finite set of places;
- T is a finite set of transitions;

- $F \subseteq (P \times T) \cup (T \times P)$ is a set of directed arcs (flow relation). They connect places to transitions and transitions to places.

Any diagram can be mapped onto such triple and vice versa.

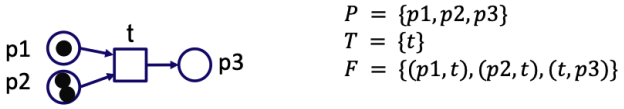


Figure 22: Formal Definition of a Petri Net

Preset and Postset

For a given transition $t \in T$:

- The **preset** of t is the set of places that have arcs leading to t:

$$\bullet t = \{p \in P | (p, t) \in F\}$$

- The **postset** of t is the set of places that have arcs leading from t:

$$t\bullet = \{p \in P | (t, p) \in F\}$$

Marking

A **marking** of a Petri net is a function $M : P \to \mathbb{N}$ that assigns to each place $p \in P$ the number of tokens $M(p)$ present in that place. For example, given the Petri net in the figure 3.2, it's marking can be represented as:

$$M(p1) = 1, M(p2) = 2, M(p3) = 0$$

or alternatively as the multi-set:

$$M = [1 \cdot p1, 2 \cdot p2]$$

Enabling and Firing

A transition $t \in T$ is **enabled** under a marking M if for every place $p \in \bullet t$, $M(p) > 0$. In other words, there is at least one token in each of its input places.

When an enabled transition t **fires**, it produces a new marking M' defined as follows:

$$M'(p) = M(p) - W(p, t) + W(t, p)$$

where $W(x, y)$ is the weight of the arc from x to y (default is 1 if not specified). This means that firing t consumes tokens from its input places and produces tokens in its output places.

Petri Net System

A **Petri net system** is a pair $S = (PN, M_0)$ where:

- PN is a Petri net (P, T, F);
- M_0 is the initial marking of the Petri net.

3.3 Mapping Petri Nets to Transition Systems

To introduce the relationship between Petri nets and transition systems, we first need to define a few concepts.

Case: a case of a Petri net is a sequence of valid transition firings that starts from the initial marking:

$$m_0 \xrightarrow{t_1} m_1 \xrightarrow{t_2} m_2 \xrightarrow{t_3} ... \xrightarrow{t_k} m_k$$

where each m_i is a marking and each t_i is a transition, m_0 is the initial marking.

Reachability Graph algorithm:

1. Label the initial marking m_0 as the root and tag it as new;
2. While there are new markings:
   - Select a new marking m;
   - If no transition are enabled in m, tag m as dead;
   - For each transition t enabled in m:
     - **Obtain** the new marking m' by firing t in m;
     - **If** m' doesn't appear in the graph yet, add it as a new marking;
     - **Add** an arc from m to m', labeled with t if not already present;
   - Tag m as old;

For example, given the Petri net in figure 3.3, the reachability graph can be obtained by:

- Create the node for the initial marking $m_0 = (p_1, free, p_4)$ and tag it as new;
- Since transition t_1 and t_4 are enabled in m_0, we can fire them:
  - Firing t_1 produces the new marking $m_1 = (p_2, free, p_4)$, we add it to the graph with the corresponding arc from m_0 to m_1 labeled t_1;
  - Firing t_4 produces the new marking $m_2 = (p_1, free, p_5)$, we add it to the graph with the corresponding arc from m_0 to m_2 labeled t_4;
- We tag m_0 as dead, since no other transitions are enabled in it;

- We select one of the new markings;
- ...

Continuing this process until no new markings are left, we obtain the reachability graph shown in figure 3.3.

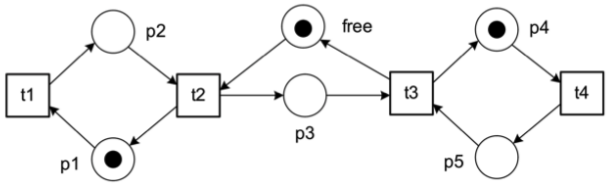


Figure 23: Petri Net Structure

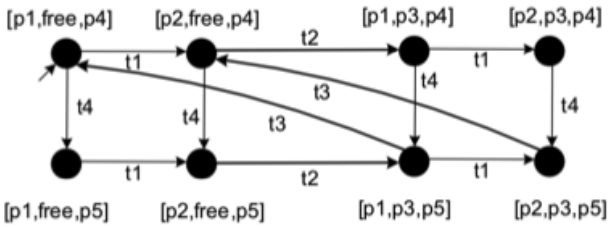


Figure 24: Reachability Graph derived from the Petri Net

State As we have seen in the example, each state in the transition graph corresponds to a marking in the Petri net. Therefore, we can derive that each state can be represented as a multi-set of places that contain tokens.

Terminal Marking It's a reachable marking in which no transition is enabled. Intuitively, it represents a dead-end state of the system: once it is reached, the execution cannot continue because there are no outgoing transitions in the transition system.

Reachable markings

Reachable Marking

A marking m is **reachable** if there exists a case that leads from m_0 to m.

3.4 Properties of Petri Nets

Given the representation of a Petri net as a transition system, we can define and analyze some important properties of Petri nets:

Boundedness Intuitively, a Petri net is bounded if there is a limit to the number of tokens that can accumulate in each place.

Boundedness

A Petri net is **k-bounded** if and only if no place p can contain more than k tokens for any reachable marking. If a Petri net is 1-bounded, it is also called **safe**.

We can derive that a Bounded Petri net has a finite number of reachable markings.

Terminating Intuitively, a Petri net is terminating if it always reach a terminal marking. This implies that every run is finite.

Terminating

A Petri net is **terminating** if and only its reachability graph is finite and has no cycles.

Deadlock-freedom Intuitively, a Petri net is deadlock-free if it is always possible to fire a transition from any reachable marking.

Deadlock-freedom

A Petri net is **deadlock-free** if and only if there is at least one transition enabled in every reachable marking, except for terminal markings.

Dead transition

Dead Transition

A transition t is **dead** if it is not enabled in any reachable marking.

Liveness Intuitively, a transition is live if it can always be fired in the future, regardless of the current marking.

Liveness of a Transition

A transition t is **live** if and only if from every reachable marking m, there exists a case that leads to a marking m' where t is enabled.

A Petri net is **live** if and only if all its transitions $t \in T$ are live.
Liveness implies deadlock-freedom, but the opposite is not necessarily true.

Home-marking

Home-marking

A marking m is a **home-marking** if it is reachable from every reachable marking.

A petri net is **reversible** if and only if its initial marking m_0 is a home-marking. This is true if and only if the reachability graph is strongly connected.

Workflow nets A Petri net is a **workflow net** (WF-net) if and only if:

- It has a single source place start (no incoming transitions) and a single sink place end (no outgoing transitions);
- Every transition is on a path from start to end.

A workflow net is **sound** if and only if its short-circuit net (where start and end are connected by a transition) is live and bounded.
Soundness ensures that the modeled process can always complete properly, starting from the initial state and reaching the final state without getting stuck or running indefinitely.

4 Process Discovery

Process discovery is a key aspect of process mining that focuses on extracting process models from event logs.

4.1 Event Logs

An event log is a collection of *cases*. Each case represents as a sequence of *events* that have occurred during the execution of a process instance. Each event typically contains attributes such as:

- **Case ID**: A unique identifier for the case. Allows to group events belonging to the same process instance.
- **Activity**: The name of the activity that was performed.
- **Timestamp**: The time at which the event occurred.
- **Additional attributes**: Such as resource, cost, etc.

4.2 Discovery challenges

Process discovery faces several challenges. The event logs may contain noise, incomplete information, or infrequent behavior that can complicate the discovery process. The discovered models need to balance between:

- **Fitness**: the discovered model should be able to replicate all the behavior observed in the event log;
- **Precision**: the discovered model should not allow for behavior that is not observed in the event log;
- **Generalization**: the discovered model should be able to generalize beyond the specific cases observed in the event log, allowing for similar but unseen behavior;
- **Simplicity**: the discovered model should be as simple as possible.

4.3 Process Mining algorithms

Several algorithms have been developed for process discovery. In this section, we will discuss some of the most prominent ones. They can be cathegorized into:

- **Direct algorithmic approaches**: These algorithms directly construct a process model from the event log without intermediate steps. Examples include the Alpha Miner;

- **Two-phase approaches**: these algorithms first extract a representation of the process behavior from the event log, and then use this representation to construct a process model. Examples include the Heuristic Miner;
- **Divide and conquer approaches**: these algorithms divide the event log into smaller segments, discover process models for each segment, and then combine these models into a single process model. Examples include the Inductive Miner.
- **Computational intelligence approaches**: these algorithms use techniques from artificial intelligence, such as genetic algorithms or neural networks, to discover process models from event logs. Examples include Genetic Process Mining.

4.3.1 Alpha Miner

The Alpha Miner is one of the earliest and most well-known algorithms for process discovery. It constructs a Petri net from an event log by identifying different types of relations between activities, such as:

- **Direct succession** (directly follows): If activity A is directly followed by activity B in the event log, we denote this as A $>_L$ B.
- **Causality**: If activity A is directly followed by activity B, but B is never directly followed by A, we denote this as A $\rightarrow_L$ B. Formally, A $\rightarrow_L$ B if A $>_L$ B and not B $>_L$ A.
- **Parallelism**: If activities A and B can occur independently and in parallel, we denote this as A $\|_L$ B. Formally, A $\|_L$ B if A $>_L$ B and B $>_L$ A.
- **Choice**: If an event log shows that either activity A or activity B can occur next, we denote this as A $\#_L$ B. Formally, A $\#_L$ B if neither A $>_L$ B nor B $>_L$ A.

To learn this relations, the Alpha Miner first constructs a footprint matrix.

Footprint Matrix: The footprint matrix is a square matrix where both the rows and columns represent the activities in the event log. The entries in the matrix indicate the relations between activities based on their occurrences in the event log. As an example, consider the following footprint matrix of the log containing activities A, B, C, D and E:

$$L_1 = \left[\langle a, b, c, d\rangle^2, \langle a, c, b, d\rangle^2, \langle a, e, d\rangle^2, \right]$$

	A	B	C	D	E
A	#	→	→	#	→
B		#	||	→	#
C			#	→	#
D				#	←
E					#

Table 1: Example of a Footprint Matrix

Only the upper triangular part of the matrix is filled, as the relations are symmetric. From the footprint matrix, the Alpha Miner identifies *causal relations* and constructs places in the Petri net accordingly.

Steps of the Alpha Miner Algorithm:

1. Identify all unique activities in the event log T_L;
2. Build the set of start activities T_I and end activities T_O. They should contain all activities that appear as the first and last activity in any trace of the log, respectively;
3. Compute pairs of activity sets (A, B) such that:
   - For every activity $a \in A$ and $b \in B$, $a \rightarrow_L b$;
   - For every pair of activities $a_1, a_2 \in A$, $a_1 \#_L a_2$;
   - For every pair of activities $b_1, b_2 \in B$, $b_1 \#_L b_2$.

   In smaller terms, all activities in set A are in causal relation with all activities in set B, and there is no direct succession between any pair of activities within the same set.
4. Try to maximize the pairs found in the previous step, i.e., if $(\{A\}, \{B\})$ is found and there exists $(\{A\}, \{C\})$, then we can merge them into $(\{A\}, \{B, C\})$ iff for every $b \in B$ and $c \in C$, $b \#_L c$; If this is possible, we replace the two pairs with the merged one;
5. For each maximal set found, create a place in the Petri net. Also, create a source place connected to all start activities in T_I and a sink place connected from all end activities in T_O.
6. For each pair (A, B), connect all activities in A to the place created for the pair, and connect that place to all activities in B.

4.3.2 Heuristic Miner

The Heuristic Miner is an extension of the Alpha Miner that aims to address some of its limitations, particularly in dealing with noise and infrequent behavior in event logs. Since it is a two-phase approach, it first constructs a *dependency*

graph from the event log, which captures the relationships between activities based on their frequencies. Based on this graph, it then constructs a process model, typically represented as a Petri net or a C-net.

Dependency Graph: The dependency graph is a directed graph where nodes represent activities, and edges represent the dependencies between them. Each edge is assigned a **weight** and a **frequency**.

Compute frequency matrix: To compute the dependency measures, the Heuristic Miner analyzes the event log and extracts the frequencies of direct successions between activities. This information are kept in a *direct succession matrix*, similar to the footprint matrix used in the Alpha Miner. This matrix records how many times each activity is directly followed by another activity in the event log.

Compute dependency measures: Using the direct succession matrix, the Heuristic Miner computes dependency measures for each pair of activities (a, b) using the formula:

$$\begin{cases} |a \implies_L b| = \frac{|a \to_L b| - |b \to_L a|}{|a \to_L b| + |b \to_L a| + 1} \\ |a \implies_L a| = \frac{|a \to_L a|}{|a \to_L a| + 1} \end{cases}$$

Where $|a \to_L b|$ is the frequency of direct succession from activity a to activity b in the event log. The dependency measure can correctly identify parallelism, causality, and choice between activities based on their frequencies.

Dependency Matrix: The dependency matrix can be computed from the frequency matrix, by applying the dependency measure formula to each pair of activities.

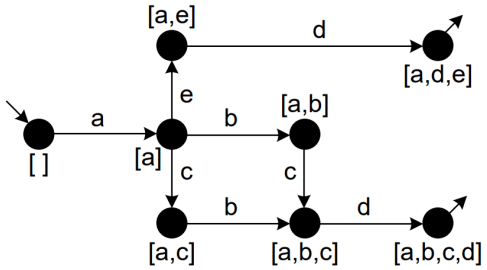
Constructing the Dependency Graph: Given the dependency matrix, and a threshold value θ, the Heuristic Miner constructs the dependency graph. For each pair of activities (a, b), if the dependency measure $|a \implies_L b|$ exceeds the threshold θ, an edge is created from node a to node b in the dependency graph.

4.3.3 Region-based mining

Region-based mining is a technique used to discover Petri nets from event logs by identifying regions in a transition system derived from the log. Can be used to discover more complex control-flow structures but it has problems dealing with noise and infrequent behavior.

Learning a transition system

A transition system is a directed graph where nodes represent states and edges represent transitions between states. In order to learn a transition system from an event log, we need to define a state representation. We can consider the past or/and future of a trace.
In this context, the **past** of a trace refers to the sequence of activities executed prior to a given current state. We construct a transition system based on the event log's traces, specifically designed to model and replay the observed behavior.
Then, we define the **future** of a trace as the sequence of activities that are expected to occur after the current state. Starting from the event log, we define the initial state as the complete set of observed traces. We then derive the state space by iteratively removing the leading activity from each trace at every transition until the traces are exhausted.
We can also combine both past and future representations to create a more comprehensive transition system.
Other possible apporaches include the use of abstractions, such as multisets (keep the frequency, not the order) or sets (only keep the activities, not the frequency nor the order), keeping only the last k activities. This allows for a more compact representation of the state space.



$$L_1 = [\langle a,b,c,d\rangle^3, \langle a,c,b,d\rangle^2, \langle a,e,d\rangle]$$

Figure 25: Example of a Past with multiset abstraction Transition System learned

Identifying regions

We assume there is only one initial state and it's convenient to also have one final state. All states need to be reachable.
A **region** is a set of states such that if a transition exits the region, then all equally labeled transitions exit the region, and if a transition enters the region, then all equally labeled transitions enter the region. All events not entering or exiting the region do not cross the region.
We can define some properties of regions:

- **Trivial regions**: regions that contain either all states or no states at all.

- **Complement**: if R is a region, then its complement (the set of states not in R) is also a region.
- **Union and intersection**: if R1 and R2 are regions, then their union (R1 $\cup$ R2) and intersection (R1 $\cap$ R2) are also regions.

We want to compute all the **minimal non-trivial regions** (i.e., the smallest possible sets of states that still satisfy the region exit/entry constraints) of the transition system for our algorithm.

Constructing the Petri net

- For each event in the transition system, a transition is created in the Petri net.
- Compute the minimal non-trivial regions of the transition system.
- For each region, a place is created in the Petri net.
- Add correspoing arcs:
  - Pre regions: if an event exits the region r (r is a pre region), an arc is added from the corresponding place to the transition.
  - Post regions: if an event enters the region r (r is a post region), an arc is added from the transition to the corresponding place.
- A token is added to each place corresponding to a region that contains the initial state of the transition system.

4.3.4 Inductive Miner

The Inductive Miner is a divide-and-conquer algorithm for process discovery that constructs process models by recursively splitting the event log into smaller segments based on identified patterns. The first step of the algorithm is to analyze the event log and compute the **directly-follows graph** (each activity is connected to the activities that directly follow it).

Partition activities

The idea is to recursively partition the set of activities in the log into subsets of the form $\langle x \rangle^k$. We can apply different **cuts** to partition the activities in the log:

- **Sequence cut** ($\rightarrow$): cutting arcs all going in the same direction.

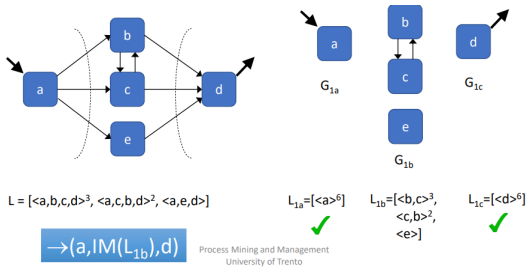


Figure 26: Example of Sequence Cut

For each of the two subsets of activities obtained from the cut, we create the sublog containing only the traces that involve activities from that subset, as shown in the figure above.

- **Exclusive choice cut** ($\times$): no arcs going one set to another

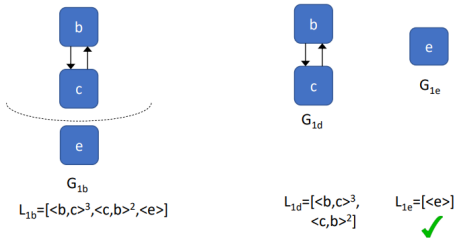


Figure 27: Example of Exclusive Choice Cut

- **Parallel cut** ($\wedge$ or $+$): each activity in one subset is connected to each activity in the other subset

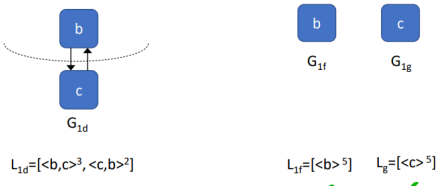


Figure 28: Example of Parallel Cut

- **Redo-loop cut** ($\circlearrowleft$): one subset is the *body* of the loop, the other subset contains the *redo* part and possibly the *exit* part.

The Inductive Miner recursively applies these cuts, trying to compute in the following order:

1. a non trivial exclusive choice cut;
2. if it is not possible, a non trivial sequence cut;
3. if it is not possible, a non trivial parallel cut;
4. if it is not possible, a redo-loop cut;
5. if none of the previous cuts is possible, it falls back to the *flower model*, which allows for any behavior.

In this way we can create a process tree representing the structure of the process, by combining the subtrees obtained from each segment of the log. Finally, the process tree is converted into a Petri net for further analysis and visualization.

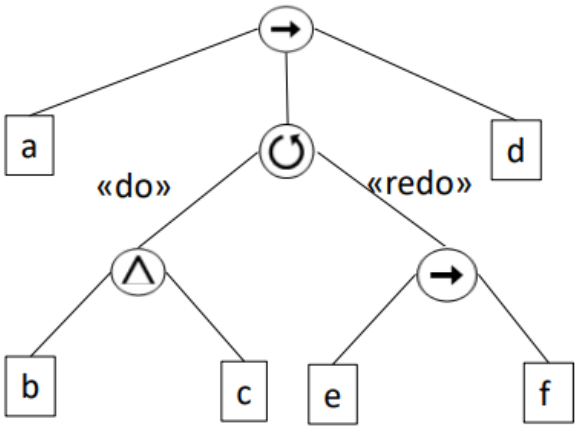


Figure 29: Example of Process Tree

5 Conformance Checking

Conformance check tries to verify if the behavior of a process model is in line with the behavior recorded in an event log. It identifies execution traces in the event log that deviate from the process model and vice versa. There are several techniques to perform conformance checking.

5.1 Replay-based Conformance Checking

Replay-based conformance checking techniques try to replay the traces recorded in an event log on a process model. During the replay, information about deviations are collected:

- **Produced Tokens** p**:** the count of tokens produced during the replay of a trace. In the beginning, a token will be produced for the source place. Each time a transition is fired, tokens will be produced for each output place of the transition;
- **Consumed Tokens** c**:** the count of tokens consumed during the replay of a trace. At the end, a token will be consumed for the sink place. Each time a transition is fired, tokens will be consumed from each input place of the transition;
- **Missing Tokens** m**:** When an activity is executed in the event log, but the corresponding transition in the process model cannot be fired due to missing tokens, tokens are artificially added to enable the firing of the transition. Note that these tokens are not considered in the produced tokens count;
- **Remaining Tokens** r**:** After replaying a trace, the tokens that are left in the process model.

Using these statistics, we can define the fitness of a trace as follows:

$$Fitness = \frac{1}{2}(1 - \frac{m}{c}) + \frac{1}{2}(1 - \frac{r}{p}) \tag{5.1}$$

This measure yields a value between 0 and 1, that can be interpreted as the percentage of behavior in the event log that can be explained by the process model.

Fitness of an Event Log The fitness equation 5.1 can be used to calculate the fitness of a single trace. We can extend this to calculate the fitness of an entire event log by summing the statistics over all traces in the log (we can compute the statistics for each unique trace and then weight them by their frequency in the log):

$$Fitness_{Log} = \frac{1}{2}(1 - \frac{\sum_{t \in L} m(t)}{\sum_{t \in L} c(t)}) + \frac{1}{2}(1 - \frac{\sum_{t \in L} r(t)}{\sum_{t \in L} p(t)}) \tag{5.2}$$

Pros and Cons: Replay-based conformance allows for a simple and intuitive way to differentiate between fitting and non-fitting traces. However, it has some limitations:

- Fitness is too high for problematic event logs;
- Only defined for Petri nets;
- No information about where the deviations occur in the process model;

5.2 Alignment-based Conformance Checking

Alignment-based conformance checking is built upon the weaknesses of replay-based conformance checking. In fact, replay-based techniques cannot provide a corresponding path in the process model.
The idea of alignment-based conformance checking is to find the best possible alignment between a trace in the event log and a corresponding path in the process model. An alignment is a sequence of steps that map events in the trace to transitions in the process model. Each step in the alignment can be one of the following:

- **Synchronous Move:** an event in the trace matches a transition in the process model;
- **Log Move:** an event in the trace does not have a corresponding transition in the process model;
- **Model Move:** a transition in the process model is executed without a corresponding event in the trace;

It's an **illegal** move to two different events between the log and the model.

Then we define a **standard cost** function to assign costs to each type of move:

- **Synchronous Move:** cost 0;
- **Log Move:** cost 1;
- **Model Move:** cost 1;

The goal of alignment-based conformance checking is to find the alignment with the lowest total cost, which is defined as the **optimal alignment**.

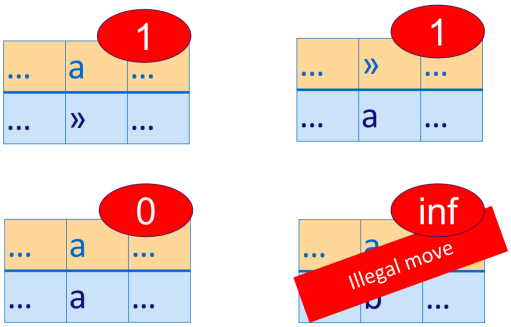


Figure 30: Example of an alignments

It's possible to define various cost functions depending on the specific application or domain, or to assign different weights to different types of moves.

Computing Fitness

The idea is to compare the cost of an optimal alignment with the cost of a worst-case alignment. In formula:

$$fitness(\sigma, N) = 1 - \frac{\delta(\lambda_{opt}(\sigma, N))}{\delta(\lambda_{worst}(\sigma, N))} \tag{5.3}$$

Where:

- σ is the trace in the event log;
- N is the process model;
- δ is the cost function;
- $\lambda_{opt}(\sigma, N)$ is the optimal alignment between the trace and the process model;
- $\lambda_{worst}(\sigma, N)$ is the worst-case alignment between the trace and the process model;

The **worst-case** alignment can be easily calculated by performing a complete sequence of log moves to represent the entire trace, followed by a series of model moves to traverse the shortest path from the initial state to the final state.

In order to compute the fitness of an entire event log, we can extend the previous formula by summing the fitness of each trace, weighted by the number of times the trace appears in the event log:

$$fitness(L, N) = 1 - \frac{\sum_{\sigma \in L} L(\sigma) \times \delta(\lambda_{opt}(\sigma, N))}{\sum_{\sigma \in L} L(\sigma) \times \delta(\lambda_{worst}(\sigma, N))} \tag{5.4}$$

where $L(\sigma)$ is the frequency of σ on the log L. Otherwise you can also use this formula, computing the fitness of each trace and then averaging it:

$$fitness(L, N) = \frac{\sum_{\sigma \in L} L(\sigma) \times fitness(\sigma, N)}{\sum_{\sigma \in L} L(\sigma)} \tag{5.5}$$